中间代码生成模块详细设计文档

71123236 张用长

1. 模块定位

```
带语义动作节点的语法树
    ↓
语义引擎（遍历、翻译）
    ↓
SemanticEngine.java / IRGenerator.java
    ↓
可执行的中间代码生成程序
    ↓
中间代码（四元式 / 三地址码 / 可扩展 Jimple）
    ↓
可选：Soot / SootUp
    ↓
Java 字节码
```

本模块的输入不是源代码，也不是 token，而是 **Yacc 模块已经构造好的 AST**。

本模块的核心任务是：

1. 做语义检查；
2. 管理符号表；
3. 把 AST 翻译成中间代码。

---

2. 输入与输出

2.1 输入

- `ASTNode root`

2.2 输出

- `List<Quad>` 四元式序列
- 语义错误信息
- 可选输出：Jimple-like IR

---

3. 设计目标

3.1 功能目标

- 支持变量声明
- 支持赋值
- 支持算术表达式
- 支持关系表达式
- 支持 `if / if-else`
- 支持 `while`
- 支持 `return`
- 支持块作用域
- 支持重复定义、未声明使用等语义检查

3.2 非功能目标

- 中间代码格式清晰
- 易于调试和测试
- 后续可扩展到 Jimple / 字节码生成

---

4. 模块内部类设计

```
semantic/
├── SemanticEngine.java
├── Symbol.java
├── Scope.java
├── SymbolTable.java
├── Quad.java
├── IRGenerator.java
├── TempFactory.java
└── LabelFactory.java
```

4.1 `SemanticEngine`

职责：

- 遍历 AST
- 检查语义正确性
- 管理作用域
- 调用 `IRGenerator` 生成中间代码

4.2 `SymbolTable`

职责：

- 进入/退出作用域
- 定义符号
- 查找符号

4.3 `IRGenerator`

职责：

- 生成四元式
- 分配临时变量
- 分配跳转标签

---

5. 数据结构设计

5.1 符号项 `Symbol`

```java
public class Symbol {
    private final String name;
    private final String type;
    private final int scopeLevel;
    private boolean initialized;

    public Symbol(String name, String type, int scopeLevel) {
        this.name = name;
        this.type = type;
        this.scopeLevel = scopeLevel;
        this.initialized = false;
    }
}
```

5.2 作用域 `Scope`

```java
public class Scope {
    private final int level;
    private final Map<String, Symbol> symbols = new HashMap<>();

    public Scope(int level) {
        this.level = level;
    }

    public boolean contains(String name) {
```

```java
        return symbols.containsKey(name);
    }

    public void define(Symbol symbol) {
        symbols.put(symbol.getName(), symbol);
    }

    public Symbol resolve(String name) {
        return symbols.get(name);
    }
}
```

5.3 符号表 `SymbolTable`

```java
public class SymbolTable {
    private final Deque<Scope> scopes = new ArrayDeque<>();

    public void enterScope() {
        scopes.push(new Scope(scopes.size()));
    }

    public void exitScope() {
        scopes.pop();
    }

    public boolean define(Symbol symbol) {
        Scope current = scopes.peek();
        if (current.contains(symbol.getName())) {
            return false;
        }
        current.define(symbol);
        return true;
    }

    public Symbol resolve(String name) {
        for (Scope scope : scopes) {
            Symbol symbol = scope.resolve(name);
            if (symbol != null) return symbol;
        }
        return null;
    }
}
```

5.4 四元式 `Quad`

```java
public class Quad {
    private final String op;
    private final String arg1;
    private final String arg2;
    private final String result;

    public Quad(String op, String arg1, String arg2, String result) {
        this.op = op;
        this.arg1 = arg1;
        this.arg2 = arg2;
        this.result = result;
    }

    @Override
    public String toString() {
        return "(" + op + ", " + arg1 + ", " + arg2 + ", " + result + ")";
    }
}
```

---

6. AST 节点约定

语义模块依赖 Yacc 输出的 AST 结构，因此必须事先约定节点类型。

建议至少包含：

- `PROGRAM`
- `FUNCTION_DEF`
- `BLOCK`
- `DECL_STMT`
- `ASSIGN_STMT`
- `IF_STMT`
- `WHILE_STMT`
- `RETURN_STMT`
- `BINARY_EXPR`
- `IDENTIFIER`
- `INT_LITERAL`

6.1 子节点顺序约定

例如：

赋值语句

```
ASSIGN_STMT
├── IDENTIFIER(a)
└── expr
```

if 语句

```
IF_STMT
├── condition
├── thenBranch
└── elseBranch(optional)
```

while 语句

```
WHILE_STMT
├── condition
└── body
```

---

7. 翻译策略

采用“**递归遍历 AST + 返回 place**”的经典翻译方案。

7.1 语句节点

- 一般不返回值
- 主要产生副作用：更新符号表或输出四元式

7.2 表达式节点

- 返回表达式结果所在位置 `place`
- 若是常量，返回常量字面值
- 若是变量，返回变量名
- 若是复合表达式，生成临时变量并返回 `tN`

---

8. 核心算法设计

8.1 声明语句

输入

```
DeclStmt(type, name, init?)
```

处理逻辑

```
1. 检查当前作用域是否已定义 name
2. 若已定义，报重定义错误
3. 否则插入符号表
4. 若有初始化表达式：
5.   p = translateExpr(init)
6.   emit(assign, p, -, name)
```

Java 伪代码

```java
private String visitDecl(ASTNode node) {
    String type = node.getAttributes().get("declType").toString();
    ASTNode idNode = node.getChildren().get(0);
    String name = idNode.getText();

    if (!symbolTable.define(new Symbol(name, type, currentScopeLevel()))) {
        throw semanticError("重复定义变量: " + name, node);
    }

    if (node.getChildren().size() > 1) {
        String place = visit(node.getChildren().get(1));
        ir.emit("assign", place, "-", name);
    }
    return null;
}
```

8.2 标识符引用

```
1. 在符号表中查找 name
2. 若找不到，报未声明错误
3. 否则返回 name
```

8.3 整数字面量

```
直接返回字面量文本
```

8.4 二元表达式

处理逻辑

```
1. p1 = translate(left)
2. p2 = translate(right)
3. t = newTemp()
4. emit(op, p1, p2, t)
5. return t
```

Java 伪代码

```java
private String visitBinaryExpr(ASTNode node) {
    String op = node.getText();
    String left = visit(node.getChildren().get(0));
    String right = visit(node.getChildren().get(1));
    String temp = ir.newTemp();
    ir.emit(op, left, right, temp);
    return temp;
}
```

8.5 赋值语句

处理逻辑

```
1. 检查左值变量是否已声明
2. place = translate(rightExpr)
3. emit(assign, place, -, name)
```

Java 伪代码

```java
private String visitAssign(ASTNode node) {
    ASTNode idNode = node.getChildren().get(0);
    String name = idNode.getText();

    if (symbolTable.resolve(name) == null) {
        throw semanticError("变量未声明: " + name, idNode);
    }

    String place = visit(node.getChildren().get(1));
    ir.emit("assign", place, "-", name);
    return null;
}
```

8.6 if 语句

处理逻辑

```
1. cond = translate(condition)
2. Lfalse = newLabel()
3. Lend = newLabel()
4. emit(jfalse, cond, -, Lfalse)
5. translate(thenBranch)
6. 若存在 else:
7.   emit(jmp, -, -, Lend)
8.   emit(label, -, -, Lfalse)
9.   translate(elseBranch)
10.  emit(label, -, -, Lend)
11. 否则:
12.  emit(label, -, -, Lfalse)
```

8.7 while 语句

处理逻辑

```
1. Lbegin = newLabel()
2. Lend = newLabel()
3. emit(label, -, -, Lbegin)
4. cond = translate(condition)
5. emit(jfalse, cond, -, Lend)
6. translate(body)
7. emit(jmp, -, -, Lbegin)
8. emit(label, -, -, Lend)
```

8.8 return 语句

```
1. place = translate(expr)
2. emit(return, place, -, -)
```

---

9. IR 生成器设计

9.1 `TempFactory`

```java
public class TempFactory {
    private int counter = 0;

    public String newTemp() {
        return "t" + (++counter);
    }
}
```

9.2 `LabelFactory`

```java
public class LabelFactory {
    private int counter = 0;

    public String newLabel() {
        return "L" + (++counter);
    }
}
```

9.3 `IRGenerator`

```java
public class IRGenerator {
    private final List<Quad> quads = new ArrayList<>();
    private final TempFactory tempFactory = new TempFactory();
    private final LabelFactory labelFactory = new LabelFactory();

    public void emit(String op, String arg1, String arg2, String result) {
        quads.add(new Quad(op, arg1, arg2, result));
    }

    public String newTemp() {
        return tempFactory.newTemp();
```

```java
    }

    public String newLabel() {
        return labelFactory.newLabel();
    }

    public List<Quad> getQuads() {
        return quads;
    }
}
```

---

10. `SemanticEngine` 遍历框架

```java
public class SemanticEngine {
    private final SymbolTable symbolTable = new SymbolTable();
    private final IRGenerator ir = new IRGenerator();

    public List<Quad> generate(ASTNode root) {
        symbolTable.enterScope();
        visit(root);
        symbolTable.exitScope();
        return ir.getQuads();
    }

    private String visit(ASTNode node) {
        return switch (node.getType()) {
            case PROGRAM -> visitProgram(node);
            case FUNCTION_DEF -> visitFunction(node);
            case BLOCK -> visitBlock(node);
            case DECL_STMT -> visitDecl(node);
            case ASSIGN_STMT -> visitAssign(node);
            case IF_STMT -> visitIf(node);
            case WHILE_STMT -> visitWhile(node);
            case RETURN_STMT -> visitReturn(node);
            case BINARY_EXPR -> visitBinaryExpr(node);
            case IDENTIFIER -> visitIdentifier(node);
            case INT_LITERAL -> visitIntLiteral(node);
            default -> null;
        };
    }
}
```

---

11. 作用域管理设计

11.1 进入块

当访问 `BLOCK` 节点时：

```
1. enterScope()
2. 依次处理块内语句
3. exitScope()
```

11.2 为什么必须这样做

因为像下面这种代码：

```
{
    int a;
    {
        int a;
        a = 1;
    }
    a = 2;
}
```

内层 `a` 和外层 `a` 是不同符号，必须用作用域栈区分。

---

12. 错误处理设计

12.1 重定义错误

同一作用域内重复声明：

```
int a;
int a;
```

12.2 未声明使用

```
a = 3;
```

12.3 类型不匹配

若后续支持更多类型，则在赋值和表达式翻译时检查。

12.4 返回值错误

若后续支持函数签名检查，则在 `return` 处比对函数返回类型。

---

13. 中间代码示例

13.1 输入程序

```
int main() {
    int a;
    int b;
    a = 3;
    b = a + 2;
    if (b < 10) {
        b = b + 1;
    }
    return b;
}
```

13.2 输出四元式

```
(assign, 3, -, a)
(+, a, 2, t1)
(assign, t1, -, b)
(<, b, 10, t2)
(jfalse, t2, -, L1)
(+, b, 1, t3)
(assign, t3, -, b)
(label, -, -, L1)
(return, b, -, -)
```

---

14. 与 Yacc 模块的接口约定

语义模块不重新解析语法， Yacc 模块已经保证：

1. AST 节点类型统一；
2. 各类节点孩子顺序固定；
3. `IDENTIFIER` 节点 text 是变量名；
4. `INT_LITERAL` 节点 text 是字面值；

5. `BINARY_EXPR` 节点 text 是运算符。

---

15. 总结

中间代码生成模块的核心不是“再做一遍分析”，而是：

- 接收 Yacc 给出的 AST；
- 在语义正确的前提下；
- 把程序结构翻译成机器无关的中间表示。